

Práctica de Laboratorio: Patrones y Principios SOLID

Sesión 3: Segregación de Interfaces (ISP) e Inversión de Dependencia (DIP)

Nombre los Estudiantes:

Alvarado Canchihuaman Hans

Solis Vilcapoma Jeremy Miguel

Rojas Lujan Fabrizio Sideral

Fecha: 29/08/2026

Ejercicio 1: El Reloj Inteligente vs Clásico

Código original:

```
class Reloj:
    def dar_hora(self): pass
    def conectar_wifi(self): pass

class RelojMecanico(Reloj):
    def dar_hora(self): print("Son las 10:00")
    def conectar_wifi(self): raise Exception("No tengo conectividad de red")
```

Principio violado: ISP (Segregación de Interfaces)

La interfaz Reloj obliga a todo reloj a implementar conectar_wifi(), pero un reloj mecánico no tiene esa capacidad. RelojMecanico se ve forzado a implementar un método que no puede cumplir y lanza una excepción. Hay que segregar la interfaz en capacidades independientes.

Solución corregida:

```
from abc import ABC, abstractmethod

class DarHora(ABC):
    @abstractmethod
    def dar_hora(self): pass

class Conectividad(ABC):
    @abstractmethod
    def conectar_wifi(self): pass

class RelojMecanico(DarHora):
    def dar_hora(self):
        print("Son las 10:00")

class RelojInteligente(DarHora, Conectividad):
    def dar_hora(self):
        print("Son las 10:00 (sincronizado)")
    def conectar_wifi(self):
        print("Conectado a la red WiFi")
```

Ejercicio 2: Sistema de Envíos E-commerce

Código original:

```
class EnvioFedex:
    def procesar_envio(self, destino):
        print(f"Enviando a {destino} por Fedex")

class TiendaOnline:
    def __init__(self):
        self.logistica = EnvioFedex()
    def despachar_compra(self, destino):
        self.logistica.procesar_envio(destino)
```

Principio violado: DIP (Inversión de Dependencias)

TiendaOnline (módulo de alto nivel) crea y depende directamente de la clase concreta EnvioFedex (módulo de bajo nivel). Si se quiere cambiar de transportista, hay que modificar TiendaOnline. Ambos deben depender de una abstracción.

Solución corregida:

```
from abc import ABC, abstractmethod

class ServicioEnvio(ABC):
    @abstractmethod
    def procesar_envio(self, destino): pass

class EnvioFedex(ServicioEnvio):
    def procesar_envio(self, destino):
        print(f"Enviando a {destino} por Fedex")

class EnvioDHL(ServicioEnvio):
    def procesar_envio(self, destino):
        print(f"Enviando a {destino} por DHL")

class TiendaOnline:
    def __init__(self, logistica: ServicioEnvio):
        self.logistica = logistica
    def despachar_compra(self, destino):
        self.logistica.procesar_envio(destino)

tienda = TiendaOnline(EnvioFedex())
```

Ejercicio 3: Panel de Electrodomésticos

Código original:

```
class Electrodomestico:
    def calentar(self): pass
    def congelar(self): pass
```

```
class HornoMicroondas(Electrodomestico):
    def calentar(self): print("Calentando la cena")
    def congelar(self): raise Exception("No puedo congelar, solo caliente")
```

Principio violado: ISP (Segregación de Interfaces)

Electrodomestico agrupa calentar() y congelar() en una sola interfaz. HornoMicroondas solo calienta, pero se ve obligado a implementar congelar() lanzando una excepción. Se deben separar en interfaces por capacidad.

Solución corregida:

```
from abc import ABC, abstractmethod

class Calentable(ABC):
    @abstractmethod
    def calentar(self): pass

class Congelable(ABC):
    @abstractmethod
    def congelar(self): pass

class HornoMicroondas(Calentable):
    def calentar(self):
        print("Calentando la cena")

class Refrigerador(Calentable, Congelable):
    def calentar(self):
        print("Modo descongelado")
    def congelar(self):
        print("Congelando alimentos")
```

Ejercicio 4: Vigilancia del Edificio

Código original:

```
class CamaraSony:
    def grabar(self):
        print("Grabando pasillo principal")

class SistemaSeguridad:
    def __init__(self):
        self.dispositivo_video = CamaraSony()
    def iniciar_vigilancia(self):
        self.dispositivo_video.grabar()
```

Principio violado: DIP (Inversión de Dependencias)

SistemaSeguridad depende directamente de la clase concreta CamaraSony. Cambiar de marca de cámara obliga a modificar SistemaSeguridad. Se debe depender de una abstracción de dispositivo de video.

Solución corregida:

```
from abc import ABC, abstractmethod

class DispositivoVideo(ABC):
    @abstractmethod
    def grabar(self): pass

class CamaraSony(DispositivoVideo):
    def grabar(self):
        print("Grabando pasillo principal")

class CamaraHikvision(DispositivoVideo):
    def grabar(self):
        print("Grabando con cámara Hikvision")

class SistemaSeguridad:
    def __init__(self, dispositivo_video: DispositivoVideo):
        self.dispositivo_video = dispositivo_video
    def iniciar_vigilancia(self):
        self.dispositivo_video.grabar()

sistema = SistemaSeguridad(CamaraSony())
```

Ejercicio 5: Usuarios del Sistema

Código original:

```
class UsuarioSistema:
    def iniciar_sesion(self): pass
    def banear_usuario(self): pass

class UsuarioEstandar(UsuarioSistema):
    def iniciar_sesion(self): print("Sesión iniciada")
    def banear_usuario(self): raise Exception("Acceso denegado, no soy admin")
```

Principio violado: ISP (Segregación de Interfaces)

UsuarioSistema obliga a todo usuario a implementar banear_usuario(), una acción administrativa.

UsuarioEstandar no debería tener ese permiso y se ve forzado a lanzar una excepción. Se deben separar las capacidades de autenticación y administración.

Solución corregida:

```

from abc import ABC, abstractmethod

class Autenticable(ABC):
    @abstractmethod
    def iniciar_sesion(self): pass

class Administrable(ABC):
    @abstractmethod
    def banear_usuario(self): pass

class UsuarioEstandar(Autenticable):
    def iniciar_sesion(self):
        print("Sesión iniciada")

class UsuarioAdmin(Autenticable, Administrable):
    def iniciar_sesion(self):
        print("Sesión de administrador iniciada")
    def banear_usuario(self):
        print("Usuario baneado")

```

Ejercicio 6: Almacenamiento de Backups

Código original:

```

class DiscoDuroLocal:
    def guardar_datos(self, archivo):
        print("Backup guardado en C:")

class GestorRespaldos:
    def __init__(self):
        self.storage = DiscoDuroLocal()
    def ejecutar_respaldo(self, archivo):
        self.storage.guardar_datos(archivo)

```

Principio violado: DIP (Inversión de Dependencias)

GestorRespaldos depende directamente de la clase concreta DiscoDuroLocal. Migrar a almacenamiento en la nube requeriría modificar GestorRespaldos. Debe depender de una abstracción de almacenamiento.

Solución corregida:

```

▶ from abc import ABC, abstractmethod

class AlmacenamientoBackup(ABC):
    @abstractmethod
    def guardar_datos(self, archivo): pass

class DiscoDuroLocal(AlmacenamientoBackup):
    def guardar_datos(self, archivo):
        print("Backup guardado en C:")

class AlmacenamientoNube(AlmacenamientoBackup):
    def guardar_datos(self, archivo):
        print("Backup guardado en la nube")

class GestorRespaldos:
    def __init__(self, storage: AlmacenamientoBackup):
        self.storage = storage
    def ejecutar_respaldo(self, archivo):
        self.storage.guardar_datos(archivo)

gestor = GestorRespaldos(DiscoDuroLocal())

```

Ejercicio 7: Controles de Vehículos

Código original:

```

class Vehiculo:
    def acelerar(self): pass
    def despegar(self): pass

class Automovil(Vehiculo):
    def acelerar(self): print("Acelerando en carretera")
    def despegar(self): raise Exception("Los autos no vuelan")

```

Principio violado: ISP (Segregación de Interfaces)

Vehiculo obliga a implementar despegar(), pero un automóvil no puede volar y se ve forzado a lanzar una excepción. Se deben separar las capacidades de aceleración terrestre y de vuelo.

Solución corregida:

```

▶ from abc import ABC, abstractmethod

class AlmacenamientoBackup(ABC):
    @abstractmethod
    def guardar_datos(self, archivo): pass

class DiscoDuroLocal(AlmacenamientoBackup):
    def guardar_datos(self, archivo):
        print("Backup guardado en C:")

class AlmacenamientoNube(AlmacenamientoBackup):
    def guardar_datos(self, archivo):
        print("Backup guardado en la nube")

class GestorRespaldos:
    def __init__(self, storage: AlmacenamientoBackup):
        self.storage = storage
    def ejecutar_respaldo(self, archivo):
        self.storage.guardar_datos(archivo)

gestor = GestorRespaldos(DiscoDuroLocal())

```

Ejercicio 8: Dashboard Meteorológico

Código original:

```

class APIAccuWeather:
    def obtener_clima_actual(self):
        return "Soleado y despejado"

class PantallaClima:
    def __init__(self):
        self.proveedor_clima = APIAccuWeather()
    def mostrar_clima(self):
        print(self.proveedor_clima.obtener_clima_actual())

```

Principio violado: DIP (Inversión de Dependencias)

PantallaClima depende directamente de la clase concreta APIAccuWeather. Cambiar de proveedor meteorológico obligaría a modificar PantallaClima. Debe depender de una abstracción del proveedor de clima.

Solución corregida:

```

▶ from abc import ABC, abstractmethod

class ProveedorClima(ABC):
    @abstractmethod
    def obtener_clima_actual(self): pass

class APIAccuWeather(ProveedorClima):
    def obtener_clima_actual(self):
        return "Soleado y despejado"

class APIOpenWeather(ProveedorClima):
    def obtener_clima_actual(self):
        return "Nublado con lluvias ligeras"

class PantallaClima:
    def __init__(self, proveedor_clima: ProveedorClima):
        self.proveedor_clima = proveedor_clima
    def mostrar_clima(self):
        print(self.proveedor_clima.obtener_clima_actual())

pantalla = PantallaClima(APIAccuWeather())

```

Ejercicio 9: Personal de Mantenimiento

Código original:

```

class Personallimpieza:
    def barrer_piso(self): pass
    def reparar_tuberias(self): pass

class Conserje(Personallimpieza):
    def barrer_piso(self): print("Limpiando el aula")
    def reparar_tuberias(self): raise Exception("Eso lo hace el gasfitero")

```

Principio violado: ISP (Segregación de Interfaces)

Personallimpieza obliga a implementar reparar_tuberias(), pero un conserje no hace trabajos de gasfitería y se ve forzado a lanzar una excepción. Se deben separar las tareas de limpieza y de plomería en interfaces distintas.

Solución corregida:


```

▶ from abc import ABC, abstractmethod

class TrabajoLimpieza(ABC):
    @abstractmethod
    def barrer_piso(self): pass

class TrabajoPlomeria(ABC):
    @abstractmethod
    def reparar_tuberias(self): pass

class Conserje(TrabajoLimpieza):
    def barrer_piso(self):
        print("Limpiando el aula")

class Gasfitero(TrabajoPlomeria):
    def reparar_tuberias(self):
        print("Reparando tuberías")

```

Ejercicio 10: Módulo de Autenticación

Código original:

```

class AutenticacionGoogle:
    def login(self, credenciales):
        print("Login validado por Google")

class AppFinanciera:
    def __init__(self):
        self.auth = AutenticacionGoogle()
    def acceder_cuenta(self, credenciales):
        self.auth.login(credenciales)

```

Principio violado: DIP (Inversión de Dependencias)

AppFinanciera depende directamente de la clase concreta AutenticacionGoogle. Agregar otro método de login (Facebook, correo, etc.) obligaría a modificar AppFinanciera. Debe depender de una abstracción de proveedor de autenticación.

Solución corregida:

1

```
from abc import ABC, abstractmethod

class ProveedorAutenticacion(ABC):
    @abstractmethod
    def login(self, credenciales): pass

class AutenticacionGoogle(ProveedorAutenticacion):
    def login(self, credenciales):
        print("Login validado por Google")

class AutenticacionFacebook(ProveedorAutenticacion):
    def login(self, credenciales):
        print("Login validado por Facebook")

class AppFinanciera:
    def __init__(self, auth: ProveedorAutenticacion):
        self.auth = auth
    def acceder_cuenta(self, credenciales):
        self.auth.login(credenciales)

app = AppFinanciera(AutenticacionGoogle())
```